

CHENRUI GAO

✉ cg52638@my.utexas.edu — 🌐 GitHub/cr-gao — 🔗 LinkedIn/Chenrui-Gao — 🌐 cr-gao.github.io

EDUCATION

University of Texas at Austin

M.S. in Computer Science

Austin, TX

Aug 2026 – May 2028 (expected)

University of Michigan, Ann Arbor

B.S.E. in Computer Engineering (GPA: 3.93/4.00)

Ann Arbor, MI

Aug 2024 – May 2026

Shanghai Jiao Tong University

B.E. in Mechanical Engineering (GPA: 3.70/4.00)

Shanghai, China

Aug 2022 – Aug 2026 (expected)

PUBLICATIONS

VAMP-MR: Vector-Accelerated Motion Planning and Execution for Multi-Robot-Arms

Philip Huang, **Chenrui Gao**, Jiaoyang Li

IROS 2026

RESEARCH EXPERIENCE

Carnegie Mellon University, ARCS Lab

*Research Intern; advised by Prof. **Jiaoyang Li** — basis of the VAMP-MR publication*

Pittsburgh, PA (Remote)

Nov 2024 – Oct 2025

- Built the planner's throughput-critical core in C++, batching forward kinematics and collision checks across robot configurations into CPU SIMD lanes with a struct-of-arrays layout, which removed motion validation as the bottleneck and made multi-arm motion generation **10–100×** faster, down to milliseconds
- Implemented the CBS-MP multi-robot motion planning framework on top of the vectorized primitives

University of California, Irvine

*Research Intern; advised by Prof. **Sven Koenig** — Topological Pathfinding with Ray-Cut CBS*

Irvine, CA

Jun 2025 – Aug 2025

- Proposed a hierarchical approach for constrained multi-agent routing: computing homotopy classes over 2D space to guide Conflict-Based Search (CBS)

OPEN-SOURCE CONTRIBUTIONS

verl-omni — *omni RL post-training* | **Committer (with merge access), 17 PRs merged** Jul 2026 – Present

- Implemented **On-Policy Distillation** for diffusion RL: teacher-anchored KL and flow-matching losses plus a frozen-teacher runtime that runs as a third forward-only worker beside actor and ref, with fit-loop hooks and fail-closed config validation
- Verified it end to end on SD3.5-Medium with an OCR reward and saw student reward rise from **0.23 to 0.62–0.71**, matching the FlowGRPO RL teacher about **5×** faster than GRPO while distillation KL fell monotonically from 4.2e-4 to 1.8e-5; covered by 150+ mutation-tested unit tests, GPU smoke runs and a 40-step training-curve reproduction
- Extended OPD to **multi-teacher** distillation that routes each sample by a configurable batch column to teachers either co-located with the actor or in standalone per-teacher resource pools scored concurrently
- Shipped an **asynchronous one-step-off teacher scheduler** for the v1 separate-async trainer that overlaps teacher scoring of batch k with the actor update of batch $k-1$: teacher-phase wait fell from **3.6s to 0.02s**, two-teacher step time from **22.1s to 19.0s**, and single-teacher step time by 16.7–23.7% at full load, while training curves matched inline scheduling within 0.004 PickScore
- Fixed a Ray rendezvous port collision across worker groups by slicing ports per group, which unblocked standalone multi-teacher pools; two teacher-config corrections in the same PR, bf16 weights and the inference micro-batch, cut the teacher phase from **7.76s to 3.13s**
- Found and fixed a v1-trainer bug in which co-located reward outputs overwrote the whole batch

- Adapted verl’s **hybrid rollout switching** to the diffusion v1 separate-async trainer, lending actor-co-located rollout replicas to generation at step end when the replay buffer runs short and reclaiming them once sampling reaches threshold: on a Wan2.2-TI2V-5B DanceGRPO recipe with 2 hybrid and 1 standalone GPU, median step time fell from **377.9s to 307.8s**, throughput rose from **0.340 to 0.414 videos/s**, per-GPU throughput beat a 4-GPU standalone baseline by about 8%, and reward was unchanged; also the first video pipeline to run on the v1 trainer
- Verified the lend–reclaim lifecycle with per-process event probes across 1+1 and 2+2 GPU layouts, including synchronized FSDP two-rank switching, with no hangs or collective errors and 15.85–21.24 GiB of rollout memory freed per reclaim; confirmed training-bound SD3.5-Medium runs make zero lends and match main’s step time
- Fixed three issues found along the way: moved the lend decision after standalone weight sync to remove a phantom-deficit regression of 9–11% in training-bound runs, replaced the register-then-queue assumption with a sync-then-register path because vLLM-Omni rejects requests while sleeping, and closed a startup race that routed warmup requests to a replica about to be reclaimed and had added 4–6s per step in 7 of 8 runs
- Consolidated the diffusion OPD end-to-end smoke suite from 7 runs to 2 on the v1 trainer while keeping coverage of both teacher routings, co-located and standalone teacher pools, and inline and one-step-off scheduling, cutting CI groups from 15 to 10
- Measured rollout–training numerical consistency across three diffusion pipelines, SD3.5-Medium, Qwen-Image-20B and BAGEL-7B, over 8 experiment arms, and shipped the resulting per-timestep monitoring as default
- Traced a replay crash and one-directional log-prob drift to a stale BAGEL step-count mapping and fixed it, cutting mean log-prob drift **157×** to bf16 noise in line with an analytical prediction
- Wired the torch profiler into both trainer fit paths and the reward-model rollout server, bringing profiled-step time from **616s to 70s** and trace size from 163MB to 32MB
- Leading **bitwise on-policy RL** for the Qwen3-Omni thinker, whose trainer and sampler are separate HF and vLLM-Omni implementations: compared them op by op and traced every log-prob difference to RMSNorm cast order, fused residual-norm, RoPE, attention kernels and MoE routing, with weights, GEMMs and the log-prob formula already identical
- Split the fix by owner into an opt-in HF-numeric mode in vLLM-Omni built from custom ops that work under torch.compile and CUDA graphs, a batch-invariant mode for verl’s FSDP engine and a raw log-prob mismatch metric: the full 48-layer MoE model reached **0 mismatched tokens** over two closed-loop RL steps with expert weights frozen, against 818 and 468 without it, at a **41%** rollout-throughput cost, so the mode is documented as RL-only; *in review*

verl — *LLM RL post-training* | **2 PRs merged**

Sep 2026 – Present

- Implemented **score centering**, an additive, hyperparameter-free correction for training–inference mismatch, in verl’s policy-gradient loss: the vLLM rollout stores the sampler’s top-128 log-probs per token, the tail is modeled from the trainer’s tail rescaled to the sampler’s tail mass, and a chunked, vocab-parallel autograd function applies the correction across all three trainer modes
- Reproduced the paper’s fp8-sampler experiment on Qwen3-0.6B over 400 steps with one seed per arm: plain policy gradient collapsed at step 72 and token-level importance sampling peaked at 0.52 held-out accuracy before falling to 0.31, while score centering stayed stable and ended at **0.56**, matching the paper’s ordering
- Landed a reviewer-proposed flat log-prob container during review, added a bitwise-equality test against vLLM’s default path and remeasured on GPU: step time fell from **19.2–29.4s to 14.1–15.9s**; the same change kept the new request field away from SGLang and TensorRT-LLM backends and preserved top-k heads across partial-rollout resume so both async modes work
- Fixed an fp8 weight-sync crash on tied-embedding models, where the quantized path skipped the alias filter and failed on the first sync, which unblocked the fp8 experiments above

vllm-omni — *omni-model serving: engine core & diffusion acceleration* | **11 PRs merged**

Aug 2026 – Present

- *Engine core*: added a scheduler-level **keep-mode pause** to diffusion stages so a busy replica can be quiesced for sleep or weight sync without aborting finished work: new batches stop at the next batch boundary and an all-rank device sync acknowledges the pause, matching vLLM’s engine semantics; median pause latency was about **0.1s**, verified on single- and multi-process backends and under Ulysses SP with an injected rank failure
- *Engine core*: fixed wake-up and sleep calls that reported success when the worker had failed, which reopened admission on an OOM’d GPU and broke the next request; failures now raise, the recorded state stays consistent, and a retry after memory frees recovers generation

- *Engine core*: fixed an orchestrator hang after a diffusion subprocess died, caused by a blocking send to a socket with no peer: with one of two replicas killed under 32 concurrent requests, hangs fell from **2 of 12 runs to 0**, and eviction now finishes within a second instead of stalling shutdown
- *Engine core*: made **prefix caching** safe under preemption in multi-stage omni pipelines by tracking a per-request delivery boundary that fills gaps and drops duplicates when a request resumes with a different cache hit: under forced preemption, the speech WER increase fell from **+0.104 to +0.014**, keeping the cache usable where it cuts time to first audio from 16.95s to 4.02s; *in progress*
- *Engine core*: fixed a FlashAttention cross-attention key-padding bug that silently mixed samples within a batch and affected every padded cross-attention diffusion model, raising real-weight output cosine from **0.9157 to 0.99996**
- *Engine core*: built a shadow-wrapper merge-on-load fast path for diffusion LoRA with fp32 delta fold-in and exact unload, bringing LoRA generation overhead from **+122% to +4.7%** while holding the PPO ratio at $1 \pm 1e-6$; *in review*
- *Acceleration*: derived a **sequence-parallel scheme for the ReLU linear attention** in SANA-Video, where Ulysses and Ring do not apply: because the KV state sums over the sequence dimension, SP needs only one all-reduce of a single $d \times d$ state, so communication is $O(d^2)$ at any sequence length; measured **1.80×** at SP2 and **3.57×** at SP2+CFG2 on $4 \times A800$; now extending it to SANA-Video 2.0, which reaches **1.90×** end to end at SP2 on $2 \times A800$ with the SP1 path bit-exact, alongside TP/CFG parallelism and CUDA-graph replay bit-exact against eager; *in progress*
- *Acceleration*: added tensor-parallel and CFG-parallel execution, measuring TP2 **1.52×**, CFG2 **2.04×** and TP2×CFG2 **3.02×**, with a 32-case correctness matrix and TP1 bit-exact against baseline
- *Acceleration*: added Cache-DiT caching with CPU offload, **1.56×** faster with peak GPU memory down **7.3 GiB**
- *Acceleration*: applied **regional torch.compile** to the MAGI-2 Preview native transformer, splitting each layer into compilable regions around the eager packed-attention and MoE-routing kernels and emulating eager bf16 rounding with precision casts; denoising step fell from **1.93s to 1.48s** at SP4/272p on A800 with per-GPU peak memory down **1.1–2.2 GiB**, and compiled-vs-eager differences sit at kernel reduction-order level, at a one-time compile cost of about 38s

sglang-omni — *Qwen3-Omni LLM serving performance* | **1 PR merged**

Aug 2026

- Rewrote the multimodal embedding merge on the thinker's **LLM prefill path** from a per-request Python loop into one batched GPU scatter: device-to-host syncs fell from **96 to 0** per batch and the merge stage ran **2.1–4.1×** faster, which cleared the path to **prefill CUDA graphs**
- Verified the rewrite three ways: 6000-seed differential fuzzing with element-wise identical output, byte-identical end-to-end serving logits, and an H800 performance matrix

PROJECTS

LLM Training Systems from Scratch (Stanford CS336 Assignments) | *PyTorch, Triton, NCCL*

2026

- Wrote **FSDP from scratch** for distributed training: backward-hook reduce-scatter, on-demand all-gather, fp32 master shards and a round-robin sharded optimizer
- Wrote a **FlashAttention Triton kernel** and a **KV cache** inside a Transformer LM built from the ground up with RoPE and a BPE tokenizer

TEACHING

Teaching Assistant, Intro to Computers & Programming (Prof. Jigang Wu)

SJTU, Fall 2023

HONORS

Dean's Honor List, University of Michigan

Dec 2024, May 2025

John Wu and Jane Sun Sunshine Scholarship

Nov 2023

SKILLS

Languages: C++, Python, Triton, MATLAB

ML Systems: PyTorch (torch.distributed, torch.compile), NCCL, Ray, vLLM, SGLang, verl, FSDP/TP/SP, LoRA, FlashAttention, Hydra, torch profiler

Systems & Tools: Linux, Git/GitHub OSS workflow, CUDA 13/cu130, ROS, OpenCV, NumPy, Pandas

Coursework: UT CS 395T (Datacenter/Cloud Co-design, ML for Systems), Stanford CS336, MIT 6.S081 (OS)